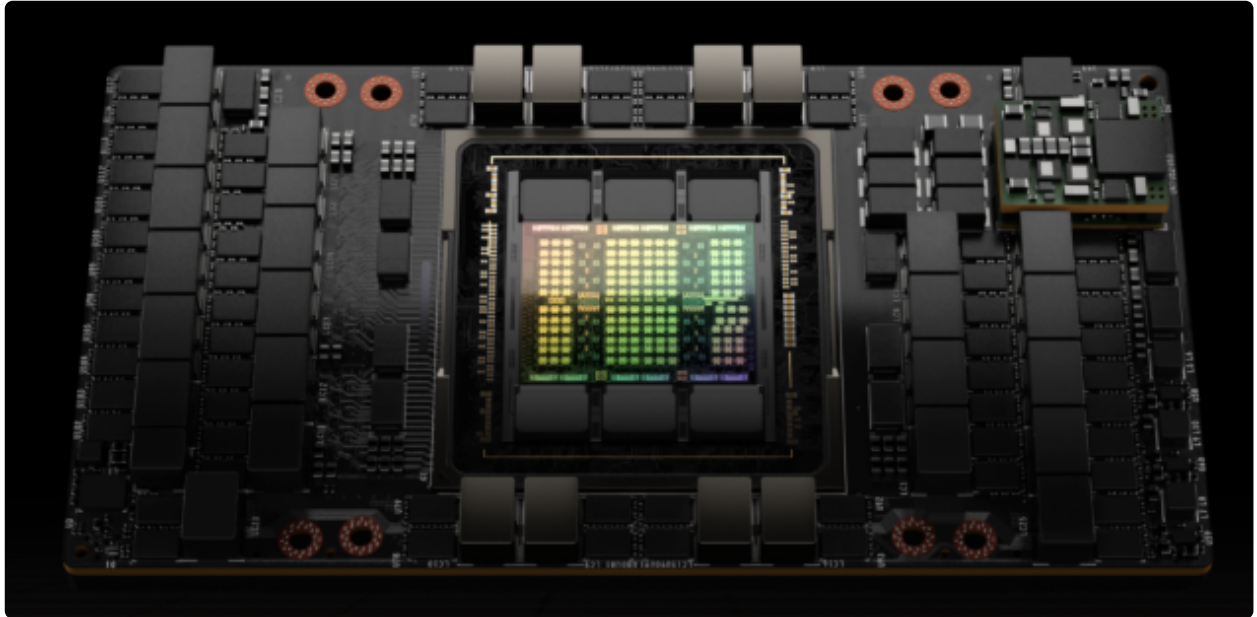


GPU 톺아보기 스터디 Week#1 : GPU 아키텍처와 병렬 연산 구조 이해

Index

1. [CPU와 GPU의 구조적 차이](#)
2. [GPU 메모리 계층 구조](#)
3. [GPU 하드웨어 및 실행 단위](#)
4. [GPU 내부 동작 흐름](#)



- [출처](#)

1. CPU와 GPU의 구조적 차이

1.1 GPU 탄생 배경

GPU(Graphics Processing Unit)는 원래 그래픽스 연산, 특히 화면의 수많은 픽셀을 동시에 처리하기 위해 태어났다. 화면을 그릴 때는 같은 종류의 연산을 빠르고 많이 수행할 수 있어야만 화면이 자연스럽게 출력되나. 그래서 많은 연산을 동시에 해내는 **Parallelism(병렬성)**이 중요하다.

시간이 지나면서 사람들은 "이 많은 연산 유닛을 그래픽 말고 일반 계산에도 쓸 수 있지 않을까?"라고 생각했고, 그 결과가 GPGPU(General Purpose GPU)다. 여기에 2006년 NVIDIA가 CUDA를 공개하며 GPU는 그래픽 카드에서 끝나지 않고, AI·시뮬레이션·과학 계산의 핵심 장치가 되었다.

GPU는 '화면 속 수많은 픽셀 수처럼 많은 연산을 동시에 처리해야 한다'는 요구에서 출발했다. 오늘날 AI에서 GPU가 강한 이유도 결국 비슷하다. AI의 기반인 행렬과 텐서 연산 역시, 같은 연산을 대량의 데이터에 반복 적용하는 문제이기 때문이다.

1.2 CPU와 GPU의 차이점

구분	CPU	GPU
설계 목표	지연 시간 최소화	처리량 최대화
코어 구성	적은 수의 강한 코어	많은 수의 단순한 코어

구분	CPU	GPU
강점	단일 작업을 빠르게 처리	같은 작업을 대량 병렬 처리
캐시 전략	큰 캐시와 복잡한 제어 로직	상대적으로 작은 캐시 + 병렬성 활용
지연 대응 방식	캐시, 분기 예측, 복잡한 제어	다른 warp로 전환하며 지연 숨기기
대표 적합 작업	운영체제, 웹 서버, 분기 많은 코드	행렬 연산, 이미지 처리, 딥러닝

CPU의 설계 철학은 "한 가지 일을 최대한 빨리 끝내는 것"이고, GPU의 설계 철학은 "동시에 많은 일을 처리한다"이다. GPU의 각 코어는 단순하지만, 많은 수의 코어가 동시에 동작한다.

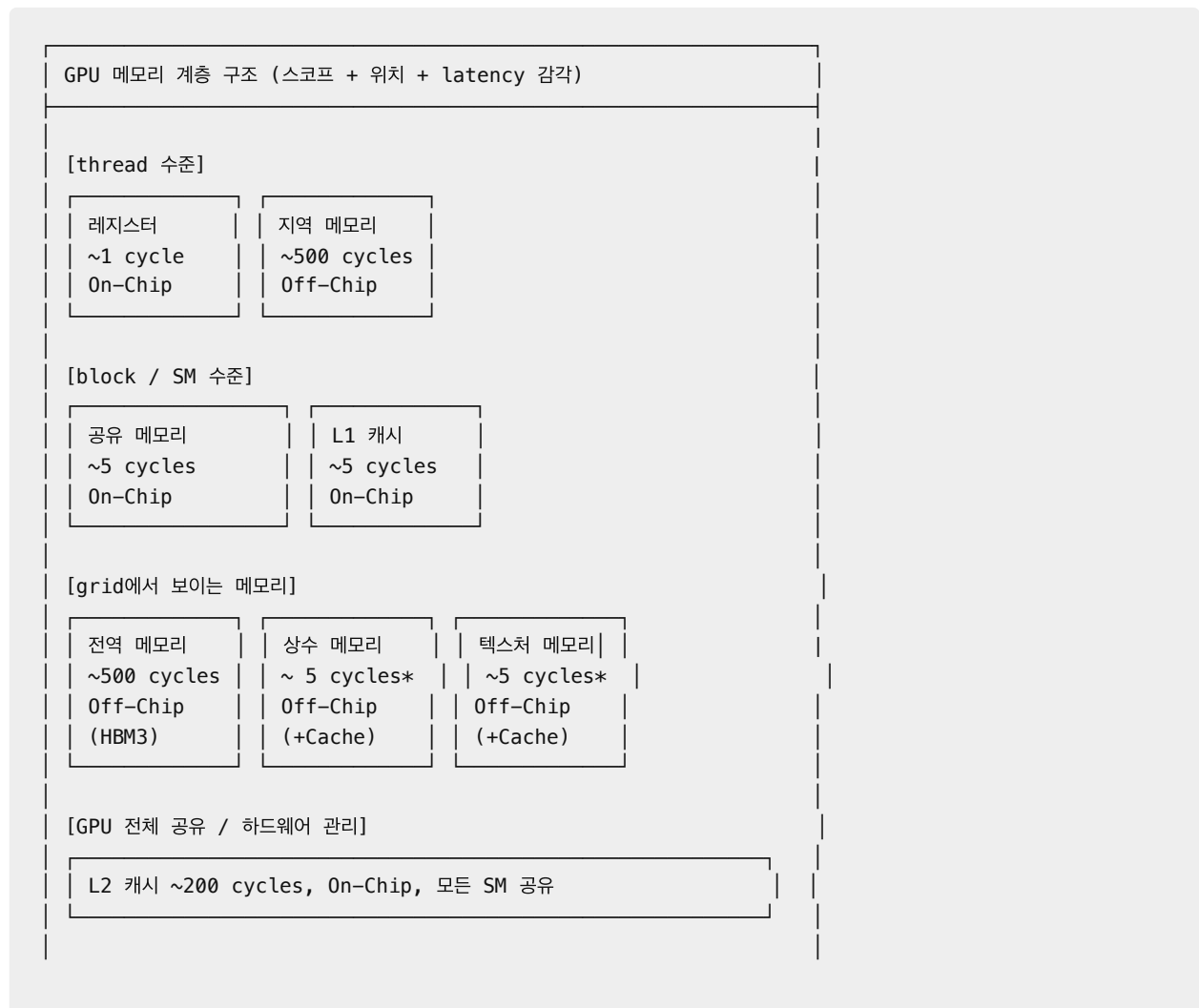
GPU가 병렬 연산에 강한 이유는 크게 네 가지다.

1. 데이터 병렬성 (**Data Parallelism**) : 같은 연산을 많은 데이터에 동시에 적용하기 쉽다.
2. 연속·규칙적인 메모리 접근 : 연속된 데이터 접근이 많으면 메모리 병합(Coalescing : 연속된 쓰레드가 연속된 메모리를 접근하면, GPU가 하나의 트랜잭션으로 묶어주는 기능)으로 대역폭을 효율적으로 활용할 수 있다. (Warp 단위 워크 플로우)
3. 지연 시간 숨기기 (**Latency Hiding**) : 일부 스레드가 메모리를 기다리는 동안 다른 warp를 돌려서 지연 시간을 숨길 수 있다.
4. 높은 메모리 대역폭 : HBM 같은 고대역폭 메모리를 통해 CPU보다 훨씬 넓은 데이터 통로를 확보한다.

2. GPU 메모리 계층 구조

GPU는 메모리가 하나만 있는 장치가 아니다. 빠르지만 작은 메모리와, 크지만 느린 메모리가 계층 구조를 이루고 있다.

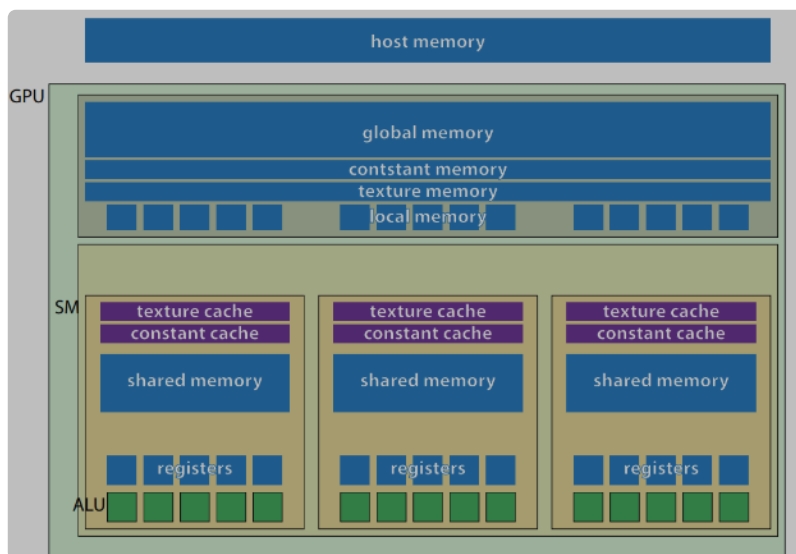
2.1 메모리 계층 구조 도식



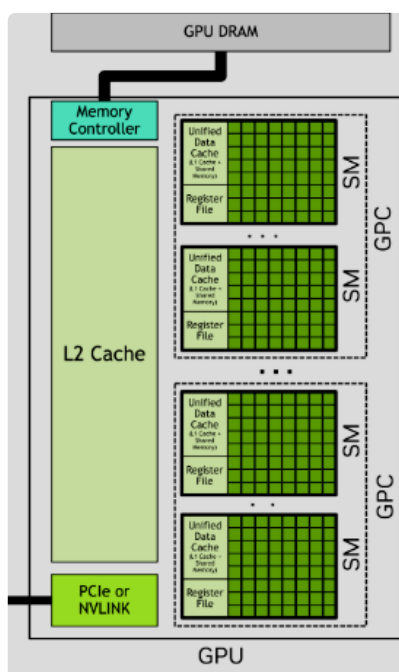
- cycle ?
 - GPU 내부 클럭이 한 번 진행되는 시간 단위를 의미한다. 각 메모리 계층 접근이 GPU 입장에 서 대략 얼마나 빨리 또는 늦게 느껴지는지를 보여주는 상대적인 표현이다.
 - 실제 cycle 수와 소요 시간은 GPU 아키텍처와 클럭, 접근 패턴에 따라 달라질 수 있다.
- *가 붙은 상수 메모리/텍스처 메모리는 전용 Cache를 가진다. Cache Hit 시 빠르고, Miss 시 전역 메모리와 비슷한 수준의 접근 속도(~ 500 Cycles)를 가진다.
- GPU에서는 같은 데이터를 여러 번 쓸 수 있다면, 느린 HBM에서 계속 읽어 오는 대신 shared memory나 register 근처로 끌어와 재사용하는 편이 훨씬 유리하다.

2.2 메모리별 역할

- [Memory Information per Compute Capability](#)



- 출처



- 출처

2.2.1 스레드 수준 메모리 (Per-Thread Memory)

스레드 수준 메모리는 각 thread 내부에서 사용되며, 다른 thread가 직접 접근할 수 없다.

레지스터 (REGISTER)

- **스cope:** Thread 전용 (다른 스레드 접근 불가)
- **위치:** SM 내부 (On-Chip)
- **레이턴시:** ~ 1 cycle (가장 빠름)
- **크기 (가장 작음)**
 - 레지스터 1개 크기 : 32-bit (4 Byte)
 - SM 당 레지스터 개수 (Hopper 기준) : 65,536개
 - SM 당 전체 레지스터 크기 : 32-bit * 65536 = 약 256KB
- **역할 및 특징**
 - CUDA Core / Tensor Core 연산을 위한 데이터를 저장하는 공간
 - CUDA 커널 내부에서 선언된 지역변수(Local Variables)가 저장된다.
 - 선언된 지역 변수 수가 할당 가능한 레지스터 수보다 많은 경우, 컴파일러가 어떤 변수를 레지스터에 저장할 지 결정한다. 초과분의 지역변수는 지역메모리(Local Memory)에 저장한다.
 - SM 내 모든 Thread Block (이하 스레드 블록)이 SM의 레지스터 풀(Pool)을 나눠 사용한다.
 - 하나의 SM에 여러 스레드 블록이 존재하면, 각 블록이 사용할 수 있는 레지스터의 양은 줄어든다.
 - 한 스레드 블록에 레지스터가 많이 할당되면, SM 내에서 동시에 머무를 수 있는 블록과 Warp 수가 줄어든다. 이는 Occupancy (점유율 - SM 내에서 동시에 상주할 수 있는 Warp의 비율)에 영향을 미친다.

지역 메모리 (LOCAL MEMORY)

- **스cope:** Thread 전용 (다른 스레드 접근 불가)
- **위치:**
 - SM 외부 (Off-Chip)
 - GPU Device Memory (DRAM, HBM 영역) 공간의 일부를 사용
- **레이턴시:** ~ 500 Cycles (전역 메모리와 동일)
- **크기 :** 스레드당 최대 512KB
- **역할 및 특징:**
 - 레지스터와 동일하게 CUDA 커널 내부에서 선언된 지역변수가 저장된다.
 - 크기 제한으로 레지스터에 담지 못하는 큰 데이터
 - 지역변수가 레지스터 수보다 많은 경우, 초과분의 변수 (Register Spill)
 - 오프-칩 메모리이기에 접근 속도는 레지스터보다 느리지만, 크기가 더 크다.
 - 컴파일러가 자동으로 사용하므로, 사람이 직접 제어하지 못한다.

2.2.2 블록 수준 메모리 (Per-Block Memory)

블록 수준 메모리는 같은 Thread block 안의 thread들이 공유한다. 여러 thread가 같은 데이터를 반복해서 써야 하는 상황에서 가장 중요해지는 계층이다. 잘 활용하면 성능 향상을 이룰 수 있다.

공유 메모리 (SHARED MEMORY)

- **스cope:** SM 내 같은 Thread block 안의 thread 간 공유
- **위치:** SM 내부 (On-Chip)
- **레이턴시:** ~ 5 cycles
- **크기 (아키텍처별로 상이)**
 - Ampere(A100 계열): SM 당 최대 164 KB
 - Hopper(H100/200), Blackwell : SM 당 최대 228 KB
- **역할 및 특징:**

- 공유 메모리는 스레드 블록 내 모든 스레드가 공유할 수 있다. 즉, 블록 내 스레드 간 데이터 공유 통로 역할을 할 수 있다. 블록 내 스레드들이 공통적으로 자주 사용하는 데이터를 공유 메모리에 저장하면, 메모리 공간 절약 및 접근 속도를 높일 수 있다.
- 공유 메모리는 같은 스레드 블록 내 스레드들 간에만 공유되는 메모리 영역이다. 즉, 서로 다른 스레드 블록 내 스레드들은 자신이 속한 스레드 블록의 공유 메모리만 사용 가능하다. 다른 스레드 블록의 공유 메모리는 접근이 불가능하다.
- 사람이 직접 채우고 직접 재사용할 수 있는 메모리 영역이다.

L1 캐시 (L1 CACHE)

- 스코프: SM 내 같은 Thread block 안의 thread 간 공유
- 위치:
 - 위치: SM 내부 (On-Chip)
- 레이턴시: ~ 5 cycles
- 크기 (Hopper 기준):
 - Hopper에서는 SM 당 L1 캐시 + 공유 메모리를 통합해서 관리한다. 이를 Unified Data Cache라고 하며, 최대 256 KB 크기를 가진다.
 - 256 KB에서 공유 메모리 영역을 제외한 나머지 영역을 L1 캐시가 사용하며, 공유 메모리의 크기는 0 ~ 228 KB이다.
- 역할:
 - 전역 메모리에서 복사한 데이터 중 자주 사용하는 데이터들이 저장된다.
 - 공유 메모리와 달리 사람이 관리할 수 없고, 하드웨어가 자동으로 관리한다.

그리드 수준 메모리 (Per-Grid Memory)

그리드 수준 메모리는 그리드 내 모든 스레드가 접근할 수 있는 메모리 영역이다. CUDA 커널을 실행하면 그리드가 생성되며, 커널이 실행하는 모든 스레드가 접근할 수 있는 메모리 영역이다.

전역 메모리(GLOBAL MEMORY)

- 스코프: 전체 스레드 접근 가능
- 위치:
 - SM 외부 (Off-Chip)
 - GPU Device Memory (DRAM, HBM 영역) 공간
- 레이턴시: ~ 500 cycles (가장 느림)
- 크기 (GPU 별로 상이)
 - H100 SXM : 80 GB
 - H200 SXM : 141 GB
- 대역폭
 - H100 SXM : 3.35 TB/s (HBM3)
 - H200 SXM : 4.8 TB/s (HBM3e)
- 역할 및 특징:
 - GPU 메모리 중 가장 큰 메모리 영역
 - CUDA 프로그램을 위한 데이터는 기본적으로 전역 메모리에 적재되며, 필요에 따라 공유 메모리나 레지스터로 복사되어 사용된다.
 - 호스트에서 접근 가능한 GPU 메모리이다. (GPU ~ 호스트 간 데이터 통신 통로 역할)
 - 호스트 코드에서 `cudaMalloc()` 을 통해 요청하여 할당받는 공간이 전역 메모리 공간이다.
 - `cudaMemcpy()` 를 통해 호스트-디바이스 간 데이터를 복사하는 것은 전역 메모리에 있는 데이터를 복사하는 것이다.

상수 메모리 (CONSTANT MEMORY)

- 스코프: Grid 내 모든 Thread가 접근 가능 (읽기 전용)
- 위치:
 - SM 외부 (Off-Chip)
 - GPU Device Memory (DRAM, HBM 영역) 공간의 일부를 사용

- 전용 Constant Cache를 가지고 있으며, Cache는 SM 내에 존재한다.
- 크기:
 - 64 KB (모든 GPU 아키텍처에서 동일)
 - Constant Cache : 8 KB (모든 아키텍처 동일)
- 역할 및 특징:
 - CUDA 커널 실행 전에 상수 메모리에 넣을 값들을 사전 선언하며, 실행 중에는 읽기 전용으로 값이 변경되지 않는다.
 - 작고 자주 읽지만 바뀌지 않는 값을 저장하는데 적합하다.

텍스처 메모리 (TEXTURE MEMORY)

- 스코프: Grid 내 모든 Thread가 접근 가능 (읽기 전용)
- 위치:
 - SM 외부 (Off-Chip)
 - GPU Device Memory (DRAM, HBM 영역) 공간의 일부를 사용
 - 전용 Texture Cache를 가지고 있으며, Cache는 SM 내(On-Chip)에 존재한다.
- 크기:
 - 고정된 메모리 크기를 가지지 않으며, 전역 메모리 내에서 작업 크기에 따라 가변적으로 할당된다.
 - Texture Cache (Hopper 기준): 28 KB ~ 256 KB
- 역할 및 특징:
 - 텍스처 메모리는 GPU의 본래 기능인 그래픽스 연산을 위해 사용되는 메모리이다.

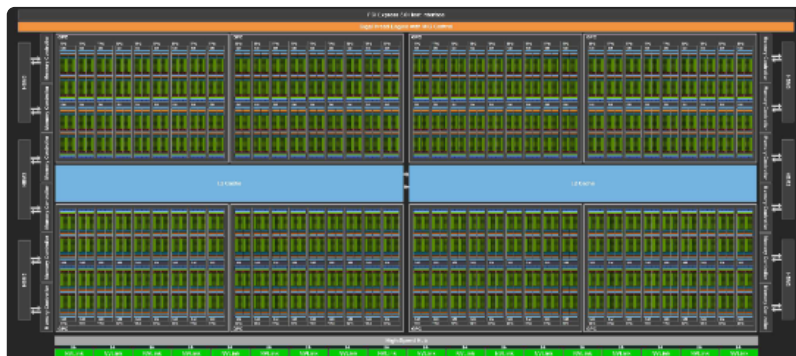
L2 캐시

- 스코프: 전체 쓰레드 접근 가능
- 위치: On-Chip
- 크기 (H100 기준): 50 MB
- 역할 및 특징:
 - 전역 메모리와 각 SM 사이에 위치하며, 모든 SM 들이 공유하는 메모리 영역이다.
 - 사람이 관리할 수 없고, 하드웨어가 자동으로 관리한다.

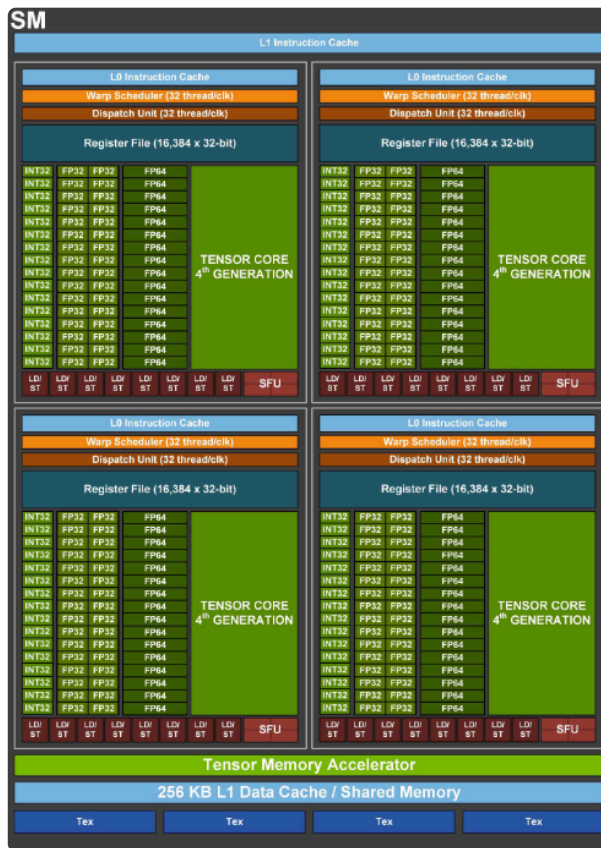
3. GPU 하드웨어 및 실행 단위

3.1 SM (Streaming Multiprocessor)

GPU는 SM(Streaming Multiprocessor)을 여러 개 묶어 놓은 구조다. 그리고 실제 실행은 Hopper 아키텍처 기준 Thread -> Warp -> Thread Block -> Thread Block Cluste -> Grid 계층을 따라 올라간다.



- 출처



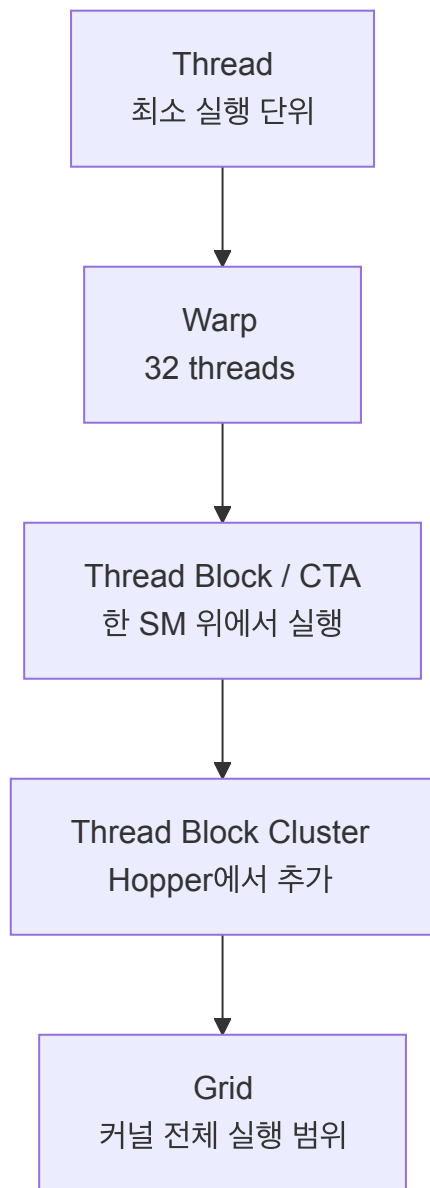
- 출처

SM은 여러 개의 CUDA 코어를 가진 연산 장치이다. H100 SXM 기준 GPU 당 132개의 SM을 탑재할 수 있으며, SM 당 CUDA 코어는 128개다. 따라서 총 CUDA Core 수는 16,896개다.

위 그림에서 볼 수 있듯이 SM 내에는 CUDA 코어 뿐만 아니라 Warp Scheduler, Tensor Code, Register File 등이 있다.

정리하자면, SM은 GPU 내부에서 연산을 수행하기 위해 Warp를 스케줄링하고, 이를 위한 메모리 및 구성요소들을 포함하는 하드웨어 블록이다.

3.2 실행 계층 구조



```
Grid
├─ Thread Block Cluster (Hopper)
│   ├── Thread Block A (SM 0)
│   │   ├── Warp 0
│   │   ├── Warp 1
│   │   └── ...
│   └─ Thread Block B (SM 1)
└─ Thread Block (단일 SM에서 실행)
```

3.2 각 단위의 의미

3.2.1 스레드 (Thread)

GPU 내의 최소 실행 단위이다. 각 스레드는 고유한 레지스터와 데이터 상태를 가진다.

3.2.2 워프 (Warp)

32개의 스레드로 구성되었으며, GPU 내에서 하나의 실제 실행 단위로 동작한다. 따라서, 워프 내 32개의 스레드가 동일한 명령을 동시에 수행한다. 이를 SIMT(Single Instruction, Multi Threads) 구조라 말한다.

스레드는 고유한 레지스터와 데이터 상태를 가지므로 워프 내에서 독립적으로 분기가 가능하지만, 이는 성능 저하를 발생시킨다.

3.2.3 스레드 블록 (Thread Block)

한 SM 위에서 실행되는 스레드 집합을 의미한다.

스레드 블록은 여러 개의 워프로 구성되었으며, 이에 따라 스레드 블록 내 총 스레드 개수는 32의 배수(128, 256, 512 ...)이다. 현재 (2026-05) 기준 모든 아키텍처에서 스레드 블록이 가질 수 있는 최대 스레드 개수는 1024개(워프 32개)이다.

동일한 스레드 블록 내의 스레드들은 공유 메모리를 통해 데이터를 공유할 수 있다.

3.2.4 스레드 블록 클러스터 (Thread Block Cluster)

Hopper 아키텍처에서 추가된 계층이다.

원래 스레드 블록은 독립적으로 실행되며, 블록 간 동기화는 불가능했다. 하지만 AI가 발전함에 따라 점점 더 큰 대규모 연산을 실행해야 됐는데, 스레드 블록 하나의 공유 메모리 크기가 작다는 한계가 있었다. 더불어 스레드 블록 간 데이터 교환을 위해서는 전역 메모리를 거쳐야만 했는데, 이는 공유 메모리에 비해 약 100배 느리다.

이를 해결하고자 여러 스레드 블록들을 물리적으로 가까운 여러 SM들에 함께 배치하여, 스레드 블록들을 협력시키는 새로운 실행 그룹을 만들었다. 스레드 블록 클러스터 내부 스레드 블록들은 동시에 실행된다. H100 기준 한 클러스터 내 최대 스레드 블록 수는 16개다.

스레드 블록 클러스터 내 블록들은 서로 다른 스레드 블록들의 공유 메모리에 접근이 가능하다. 이를 위해 서로 다른 SM에 존재하는 공유 메모리를 논리적으로 연결한 공유 메모리 구조를 가지며, 이를 분산 공유 메모리(Distributed Shared Memory, DSMEM)이라고 한다. 이 덕분에 속도가 느린 전역 메모리를 거치지 않고, 공유 메모리 간 데이터 교환으로 성능을 높일 수 있다. 분산 공유 메모리 크기는 사람이 제어할 수 있다.

3.2.5 그리드 (Grid)

하나의 CUDA Kernel Launch를 구성하는 모든 스레드 블록들의 집합을 의미한다. 즉, 커널을 실행하면 그리드가 생성된다.

4. GPU 내부 동작 흐름

4.1 전체 흐름

1. 데이터 준비
 - 출력 크기 / 문제 크기 파악
- ↓
2. 그리드 결정
 - 전체 일을 몇 개의 block으로 나눌지 결정
- ↓
3. 블록 분할
 - block 크기와 thread 수 결정
- ↓
4. SM 할당
 - block들을 사용 가능한 SM에 자동 배치
- ↓
5. 워프 실행
 - SM 안에서 warp 단위로 실제 실행

정리하면 문제 크기를 보고 **launch** 구성을 정한 뒤, GPU가 **block**을 SM에 배치하고 **warp** 단위로 실행한다.

4.2 단계별로 이해하기

4.2.1 데이터 준비

가장 먼저 해야 할 일은 문제 크기와 출력 모양을 정하는 것이다. GPU는 마법처럼 "알아서 병렬화"하지 않는다. 프로그래머가 먼저 "최종 결과가 몇 개 필요한가"를 정해야 한다.

예를 들면, 최종 결과에 대한 기준은 다음과 같다.

- 벡터 덧셈이라면 결과 벡터 길이 N
- 행렬 곱셈이라면 결과 행렬의 행 $\times$ 열
- 이미지 처리라면 출력 픽셀 수

4.2.2 그리드 결정

다음으로는 전체 일을 몇 개의 **block** 묶음으로 나눌지 정한다. 이것이 grid를 정하는 단계다.

CUDA에서 `<<<grid, block>>>`의 `grid`는 "kernel 전체가 몇 개 block으로 구성되는가"를 뜻한다.

- 데이터가 많으면 `grid`도 커진다.
- `block` 하나로 전체를 처리할 수 없으면 `block`을 여러 개 만든다.
- 결국 `grid`는 전체 문제 범위를 GPU 전체에 어떻게 펼칠지를 정하는 단계다.

4.2.3 블록 분할

이제 스레드 블록 크기, 즉 스레드 블록 1개에 스레드를 몇 개 둘지 정한다. 이는 GPU 하드웨어 효율과 직접 연결된다.

주요 고려사항은 다음과 같다.

- warp 크기(32)의 배수인가?
- shared memory를 얼마나 쓸 것인가?
- register pressure가 너무 커지지 않는가?
- block이 너무 작아서 GPU를 덜 채우는 건 아닌가?

4.2.4 SM 할당

앞선 단계를 정의하고, CUDA 커널을 실제로 실행하면, GPU는 스레드 블록들을 사용 가능한 SM들에 자동으로 배치(**block placement**) 한다. 하드웨어가 자원 상태를 보고 자동으로 배치한다.

- 하나의 스레드 블록은 하나의 SM에서 실행된다.
- 하지만 한 SM에 스레드 블록이 하나만 있는 것은 아니다. 자원이 허용하면 여러 스레드 블록이 동시에 올라갈 수 있다.

4.2.5 워프 실행

스레드 블록 SM에 올라가면, 그 안의 스레드들은 32개씩 **warp**로 묶여 실행된다. 그리고 SM 안의 Warp scheduler가 실행 가능한 Warp를 골라 연산을 수행하도록 큐에 밀어넣는다.

- 어떤 warp가 바로 실행 가능한지 본다.
- 어떤 warp가 메모리를 기다리면 다른 warp를 실행한다.
- 이런 식으로 GPU는 memory latency를 숨기며 처리량을 유지한다.

References

- [NVIDIA Developer Tech Blog | NVIDIA Hopper 아키텍처 심층 분석하기](#)
- [NVIDIA | NVIDIA H100 Tensor Core GPU Architecture](#)

- [NVIDIA | NVIDIA H100 GPU Whitepaper](#)
- NVIDIA | CUDA Programming Guide
 - [1.2 Programming Model - GPU Hardware Model](#)
 - [5.1. Compute Capabilities - Shared Memory Capacity](#)
- [NASA | Basic on nvidia gpu hardware architecture](#)